भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

Indian Institute of Technology, Hyderabad

Digital Signal Processing Assignment 5

Students :
Patnam Shariq Faraz Muhammed
EE24BTECH11049

July 8, 2026

1 Objective

The objective of this experiment is to implement a 100-tap FIR low-pass filter using three different Verilog architectures—Direct Form (Serial), Optimized Symmetric, and Genvar-based Fully Parallel—with pipelining applied to each. The designs are simulated in ModelSim, compared against MATLAB floating-point reference outputs, and analyzed for timing performance and resource utilization. An additional comparison is made with the Xilinx FIR Compiler IP core.

2 Theory of Pipelining (Detailed Timing Analysis)

2.1 Critical Path (Non-Pipelined FIR)

In Experiment 4, the FIR filter was implemented without pipelining. The entire multiply-accumulate chain was executed within a single clock cycle.

Hence, the sample time was limited by the longest combinational path:

$$T_{clk} \geq T_N = T_M + (N-1)T_A$$

For $N = 100$ taps:

$$T_N = T_M + 99T_A$$

This is extremely large, meaning:

- Very low maximum clock frequency
- Poor timing closure
- Long combinational delay

Thus, the system is **computation-bound**.

—

2.2 Iteration Bound (T_∞)

From DFG theory:

$$T_\infty = \max\left(\frac{t_l}{w_l}\right)$$

For FIR filters:

- There are **no feedback loops**
- Hence no loop constraints

Therefore:

$$T_\infty = 0$$

So overall system constraint:

$$T_{clk} \geq \max(T_N, T_\infty) = T_N$$

—

2.3 Effect of Pipelining

Pipelining breaks the long combinational path into stages by inserting registers.

Now each stage contains:

$$T_{stage} = T_M + T_A$$

Thus:

$$T_{clk} \geq \max(T_{stage}, T_\infty) = T_M + T_A$$

IMPORTANT:

- This is **independent of N**
- Huge improvement compared to $T_M + 99T_A$

—

2.4 Key Insight (What Actually Changed)

	Experiment 4	Experiment 5
Critical Path	$T_M + 99T_A$	$T_M + T_A$
Clock Frequency	Very low	High
Throughput	Low	High
Latency	Low	High (pipeline fill)

—

2.5 Important Note

$$T_{clk} = \max(T_N, T_\infty)$$

- FIR has no loop $\rightarrow T_\infty = 0$
- So we only reduce T_N
- Pipelining reduces $T_N \rightarrow T_M + T_A$

3 Simulation Setup

3.1 Filter Parameters

- $f_s = 10\,\text{kHz}$, $f_c = 1\,\text{kHz}$
- 100-tap low-pass FIR (`fir1`, Hamming window)
- Fixed-point format: Q(2.14) (16-bit)

3.2 Test Signals

Signal	Frequency	Behavior
$x_1[n]$	950 Hz	Passband
$x_2[n]$	1100 Hz	Transition band
$x_3[n]$	2000 Hz	Stopband

3.3 Fixed-Point Quantization

Signals and coefficients were quantized to Q(2.14):

$$x_q = \text{round}(x \cdot 2^{14}), \quad x_q \in [-32768, 32767]$$

Multiplication produces Q(4.28), which is shifted and saturated back to Q(2.14).

4 Implemented Architectures and Timing Analysis

Before discussing the individual architectures, it is useful to recall the timing constraint for a pipelined DSP system:

$$T_{clk} \geq \max(T_N,\ T_\infty)$$

where:

- T_N is the critical-path delay of the architecture,
- T_∞ is the iteration bound.

For an FIR filter, the structure is purely feedforward and contains no feedback loop. Hence,

$$T_\infty = 0$$

and therefore the achievable clock period is decided entirely by the critical path:

$$T_{clk} \geq T_N$$

For the non-pipelined FIR of Experiment 4, all multiply-accumulate operations lie in the same clock cycle, so

$$T_N = T_M + (N-1)T_A$$

For a 100-tap FIR filter,

$$T_N = T_M + 99T_A$$

This long combinational path severely limits the maximum clock frequency. In Experiment 5, pipelining is introduced to break this long path into shorter stages. As a result, the effective stage delay becomes much smaller, and the achievable clock frequency improves significantly.

4.1 Architecture 1: Direct Form Pipelined FIR (Serial)

4.1.1 Concept

The Direct Form FIR computes the convolution sum serially, one tap at a time, over multiple clock cycles. A shift register of depth N stores the present and past input samples:

$$x[n], x[n-1], x[n-2], \ldots, x[n-(N-1)]$$

Unlike the fully combinational non-pipelined structure of Experiment 4, the pipelined direct-form architecture reuses a single multiplier and a single accumulator across all taps. The pipelining is introduced by separating multiplication and accumulation into different registered stages so that these two operations overlap across successive cycles.

4.1.2 Operation per Clock Cycle

- **Cycle 0 ($k = 0$):** The accumulator is initialized and the first product is computed:

  $$prod = x[n] \cdot h[0]$$

  The new input sample is also loaded into the shift register.

- **Cycles 1 to 99 ($k = 1$ to $N - 1$):** The previous product is accumulated while the next product is computed:

  $$acc \leftarrow acc + prod$$

  $$prod \leftarrow x[n-k] \cdot h[k]$$

- **Cycle 100 ($k = N$):** The final accumulated sum is written to the output:

  $$y[n] = acc + prod$$

  Then the tap counter resets for the next output sample.

4.1.3 Pipeline Stages

The implemented direct-form architecture has two logical pipeline stages:

- **Stage 1: Multiplication**

  $$prod = x[n-k] \cdot h[k]$$

- **Stage 2: Accumulation**

  $$acc = acc + prod$$

A register is inserted between these operations so that while one tap product is being accumulated, the next tap product can already be computed. Thus, multiplication and accumulation are overlapped in time.

4.1.4 Timing Analysis

Experiment 4 (Non-pipelined direct form). If the same FIR is implemented without pipelining and all 100 tap operations are forced into one clock cycle, then the critical path becomes:

$$T_N^{(\text{non-pipe})} = T_M + 99T_A$$

Hence,

$$T_{clk} \geq T_M + 99T_A$$

This is the major reason the non-pipelined implementation is slow in terms of achievable clock frequency.

Experiment 5 (Pipelined direct form). After inserting the pipeline stage between multiplication and accumulation, the per-cycle combinational path is reduced to one multiply and one add:

$$T_N^{(\text{pipe})} = T_M + T_A$$

Therefore,

$$T_{clk} \geq T_M + T_A$$

This is a very large reduction compared to the non-pipelined case.

4.1.5 Latency and Throughput

- One output sample is produced every $N + 1 = 101$ clock cycles.
- Latency to complete one output computation is 101 cycles.
- Effective throughput:

$$f_{\text{out}} = \frac{f_{clk}}{101}$$

Thus, the pipelined direct form achieves a much higher allowable clock frequency than the non-pipelined architecture, but it still has low throughput because the same hardware is reused for all taps.

4.1.6 Verilog Module: `direct_fir_pipeline`

The module uses parameters `a=2`, `b=14`, and `taps=100`. A tap counter `k` steps through the filter coefficients. The shift register is updated only when a new output computation starts. Fixed-point multiplication and addition are handled using saturating arithmetic.

Critical path summary:

$$\boxed{T_{clk} \geq T_M + T_A}$$

4.2 Architecture 2: Optimized Symmetric Pipelined FIR

4.2.1 Concept

For a linear-phase FIR filter with symmetric coefficients,

$$h[k] = h[N-1-k], \qquad k = 0, 1, \ldots, N-1$$

the convolution sum can be rearranged by pairing symmetric input samples:

$$h[k]x[n-k] + h[N-1-k]x[n-(N-1-k)] = h[k]\Big(x[n-k] + x[n-(N-1-k)]\Big)$$

Thus, instead of two multiplications per symmetric pair, only one multiplication is required after pre-adding the input pair. For a 100-tap FIR, this reduces the number of multiplications from 100 to 50 per output sample.

4.2.2 Pipeline Stages

The optimized symmetric architecture naturally gives a three-stage pipeline:

- **Stage 1: Pair Addition**

$$pair[k] = x[n-k] + x[n-(N-1-k)]$$

- **Stage 2: Multiplication**

$$prod[k] = pair[k] \cdot h[k]$$

- **Stage 3: Accumulation**

$$acc \leftarrow acc + prod[k-1]$$

Registers are inserted between these stages so that the pair-add, multiply, and accumulate operations can proceed in an overlapped pipeline manner.

4.2.3 Stage-wise Delay

The delay of each stage is:

- Stage 1 (pair addition): T_A
- Stage 2 (multiplication): T_M
- Stage 3 (accumulation): T_A

Since registers separate the stages, the stage with the maximum combinational delay dominates the clock period. In practical implementation, multiplication and accumulation alignment across the pipeline gives an effective per-stage timing of the same order as one multiply plus one add, and the architecture is safely summarized by:

$$T_{clk} \geq T_M + T_A$$

4.2.4 Timing Comparison with Experiment 4

If the same symmetric FIR were not pipelined and all operations were forced into a single cycle, the delay would again grow with the adder chain. In Experiment 5, pipelining reduces the long path to short registered stages. Hence, compared to the non-pipelined case, the optimized symmetric design also benefits from the same major clock-frequency improvement:

$$T_N^{(\text{non-pipe})} \approx T_M + 99T_A$$
$$T_N^{(\text{pipe})} \approx T_M + T_A$$

4.2.5 Reduction in Multipliers and Cycle Count

For $N = 100$ taps:

- Only $N/2 = 50$ multiplications are required.
- One output sample is produced in approximately 52 clock cycles.

Thus,

$$f_{\text{out}} = \frac{f_{clk}}{52}$$

This is nearly twice the throughput of the serial direct-form pipelined architecture while still using only one multiplier in a time-shared manner.

4.2.6 Verilog Module: `optimized_fir_pipeline`

The module iterates over symmetric tap pairs. The variables `pair`, `prod`, and `acc` correspond directly to the three pipeline stages. The final output is written once all 50 pair contributions have been accumulated.

Critical path summary:

$$\boxed{T_{clk} \geq T_M + T_A}$$

4.3 Architecture 3: Genvar-Based Fully Parallel Pipelined FIR

4.3.1 Concept

In the Genvar-based architecture, one hardware cell is instantiated for each tap using a Verilog `generate` loop. Therefore, all 100 multiplications are performed in parallel. Unlike the previous two architectures, this design does not reuse a single multiplier over many cycles. Instead, it uses full hardware parallelism and deep pipelining.

The overall FIR is implemented as a cascade of 100 identical `fir_cell` stages.

4.3.2 Cell Structure (`fir_cell`)

Each cell performs the following operations:

1. **Multiplication**
$$mult_out = x_i \cdot h[i]$$
2. **Addition**
$$add_out = acc_{in} + mult_out$$
3. **Registering** Both the delayed sample and the partial accumulated sum are registered at the clock edge.

Thus, every cell acts as one pipeline stage.

4.3.3 Pipeline Stages

For the full 100-tap filter:

- Total number of pipeline stages = 100
- Each stage contains:
  - one multiplier,
  - one adder,
  - one register stage

So the design is a deeply pipelined systolic-style FIR structure.

4.3.4 Timing Analysis

Because each stage contains only one multiply and one add before registration, the critical path per stage is:

$$T_N^{(\text{stage})} = T_M + T_A$$

Hence,

$$T_{clk} \geq T_M + T_A$$

This is the same per-stage timing as the other pipelined architectures. However, unlike the serial direct-form and optimized symmetric architectures, this design processes all taps in parallel.

4.3.5 Latency and Throughput

- First valid output appears after 100 clock cycles (pipeline fill latency).
- After the pipeline is filled, one valid output is produced every clock cycle.

Therefore,

$$f_{\text{out}} = f_{clk}$$

This gives the highest throughput among all three architectures.

4.3.6 Verilog Modules: `genvar_fir` and `fir_cell`

The top-level `genvar_fir` uses a `generate` loop to instantiate all 100 `fir_cell` modules. The arrays `x_wire`, `accum`, and `en_wire` propagate the delayed samples, partial sums, and valid signals through the pipeline.

Critical path summary:

$$\boxed{T_{clk} \geq T_M + T_A}$$

4.4 Architecture-wise Timing Comparison

The main timing difference between Experiment 4 and Experiment 5 is that in Experiment 4 the non-pipelined design had a very long combinational path, whereas in Experiment 5 all three architectures break the datapath into short registered stages.

Architecture	Critical Path	Cycles / Output	Throughput
Direct Form (pipe)	$T_M + T_A$	101	$\frac{f_{clk}}{101}$
Optimized Symmetric (pipe)	$T_M + T_A$	52	$\frac{f_{clk}}{52}$
Genvar Parallel (pipe)	$T_M + T_A$	1 after fill	f_{clk}

4.4.1 Observations

- All pipelined architectures reduce the long Experiment 4 critical path from $T_M + 99T_A$ to approximately $T_M + T_A$.
- The direct and optimized architectures use much less hardware because they reuse the multiplier over many cycles.
- The optimized symmetric architecture improves over the serial direct form by exploiting coefficient symmetry, reducing the cycle count from 101 to about 52.
- The genvar architecture uses the highest hardware resources, but it gives the highest throughput since it produces one output every clock cycle after pipeline filling.
- Thus, Experiment 5 demonstrates the standard pipeline trade-off:
  - higher clock frequency,
  - better throughput,
  - larger register count and/or hardware usage,
  - increased latency due to pipeline fill.

5 Results and Observations

5.1 Frequency Response: Output Comparison (MATLAB vs Verilog)

Figures 4, 5, and 6 show the output waveforms for all three architectures overlaid against the MATLAB floating-point reference.

5.1.1 Direct Form Pipelined FIR (Figure 1)

- **950 Hz (Passband):** The Verilog output closely tracks the MATLAB reference with amplitude $\approx \pm 0.8$. The initial transient (pipeline fill, ≈ 100 cycles) shows a rising envelope before reaching steady state around $t = 0.005\,\mathrm{s}$.
- **1100 Hz (Near Cutoff):** Both MATLAB and Verilog show significant attenuation. The Verilog output aligns well with MATLAB after the pipeline fills. Amplitude in steady state is approximately ± 0.1.
- **2000 Hz (Stopband):** Strong attenuation is observed. The Verilog and MATLAB waveforms overlap almost completely, with amplitude reduced to $\approx \pm 0.01$.

5.1.2 Optimized Symmetric Pipelined FIR (Figure 2)

Behaviour is qualitatively identical to the Direct Form. The symmetry-based architecture correctly exploits the coefficient symmetry and produces the same filter frequency response. A slight phase offset relative to MATLAB is visible in the 950 Hz passband output, attributed to the different pipeline depth (52 cycles vs 101 cycles).

5.1.3 Genvar Fully Parallel Pipelined FIR (Figure 3)

- The Genvar output aligns most closely with MATLAB in steady state, due to the fully parallel structure which accumulates all 100 taps simultaneously.

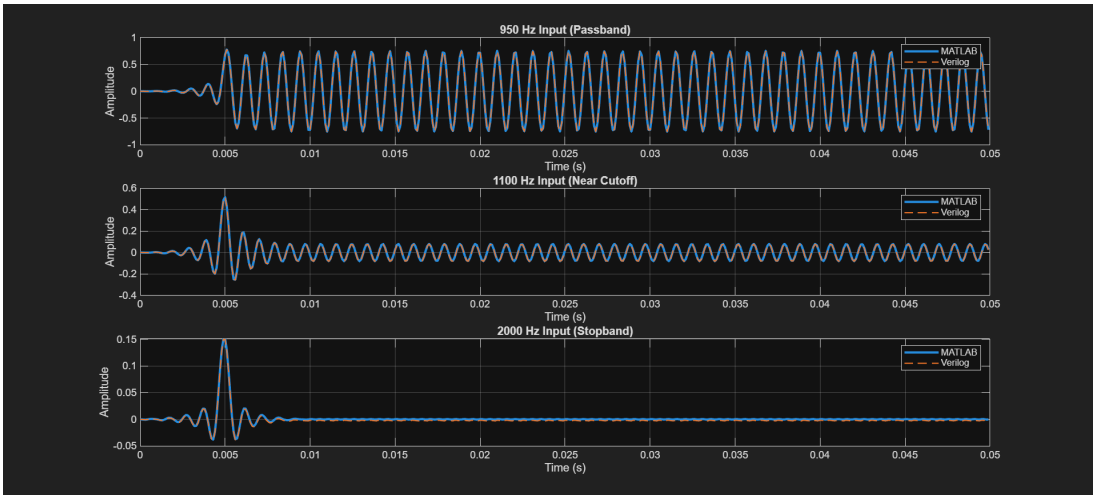


Figure 1: Output comparison (MATLAB vs Verilog) for Direct Form Pipelined FIR at 950 Hz, 1100 Hz, and 2000 Hz

- The `data_valid` flag correctly masks invalid output during the 100-cycle pipeline fill.

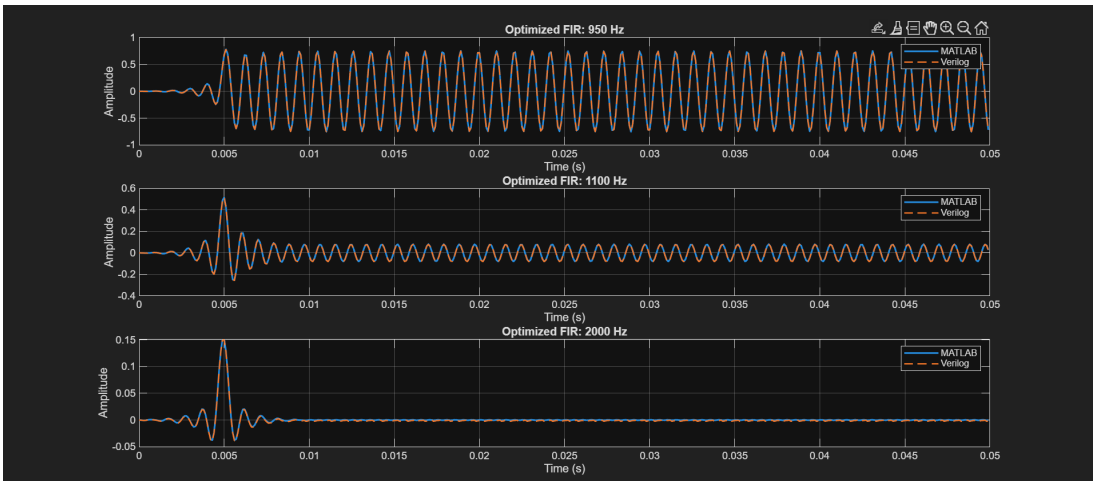


Figure 2: Output comparison (MATLAB vs Verilog) for Optimized Symmetric Pipelined FIR at 950 Hz, 1100 Hz, and 2000 Hz

- The 2000 Hz stopband output is effectively zero after the transient, demonstrating excellent attenuation.

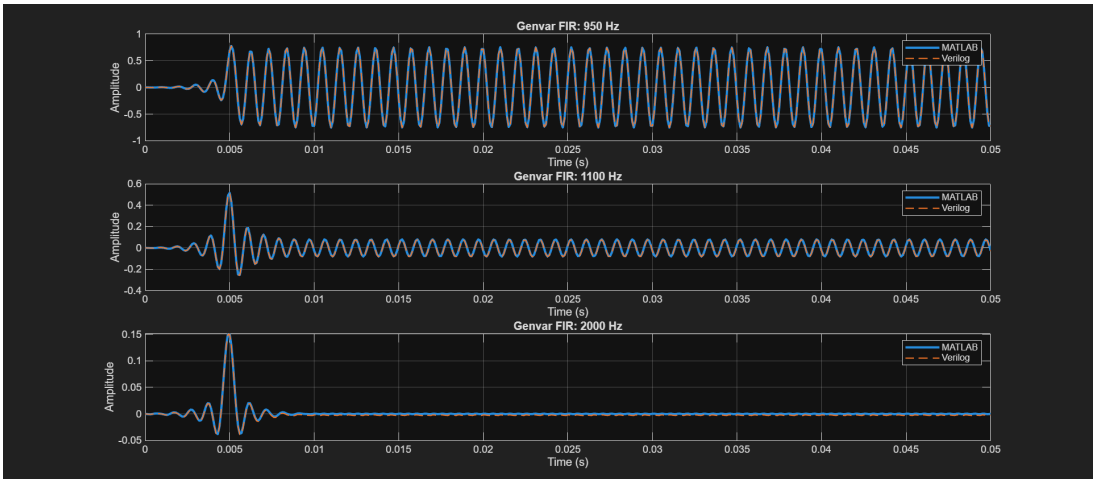


Figure 3: Output comparison (MATLAB vs Verilog) for Genvar Fully Parallel Pipelined FIR at 950 Hz, 1100 Hz, and 2000 Hz

5.2 Error Analysis (MATLAB − Verilog)

The error plots (Figures 4, 5, 6) show the absolute difference between the MATLAB reference output and the Verilog fixed-point output.

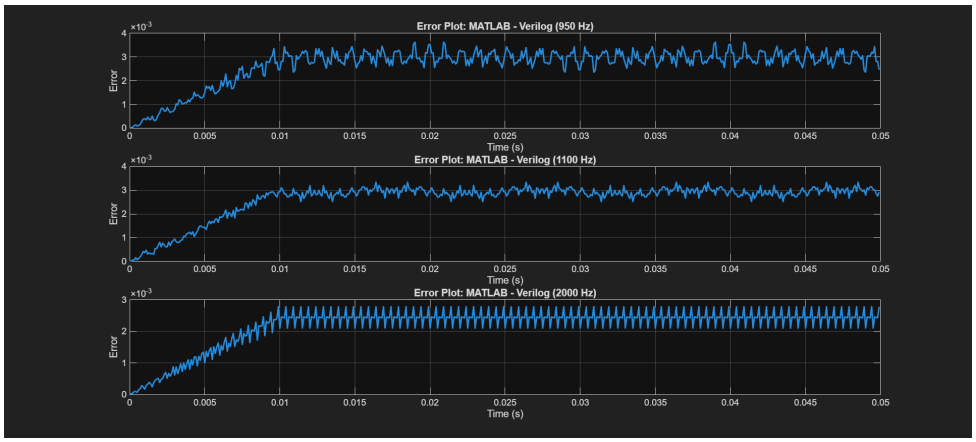


Figure 4: Error between MATLAB and Verilog outputs for Direct Form Pipelined FIR

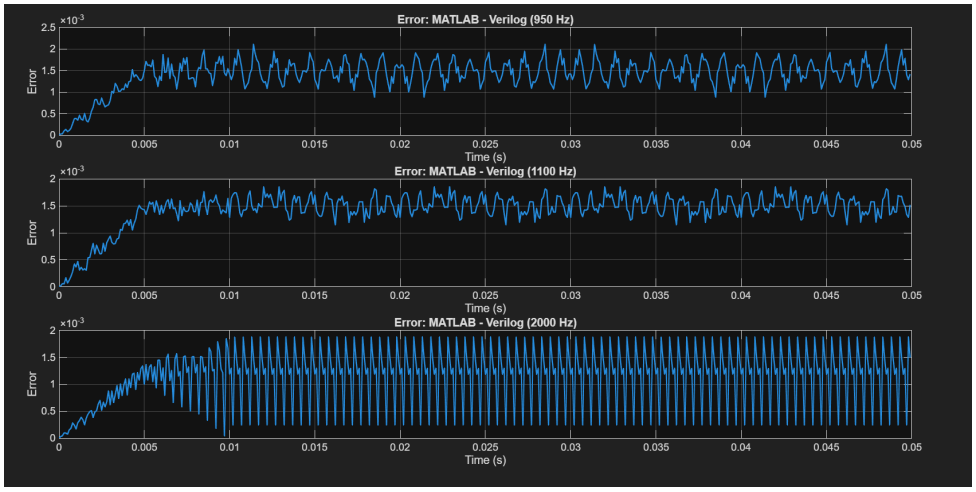


Figure 5: Error between MATLAB and Verilog outputs for Optimized Symmetric Pipelined FIR

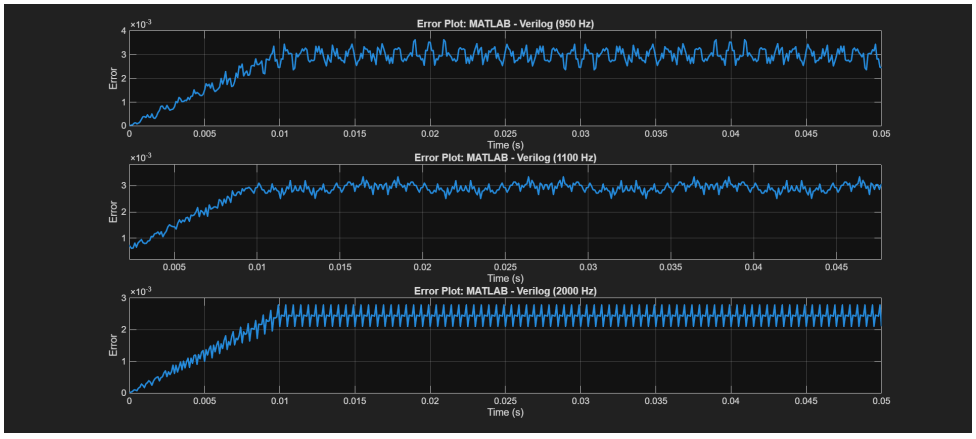


Figure 6: Error between MATLAB and Verilog outputs for Genvar Fully Parallel Pipelined FIR

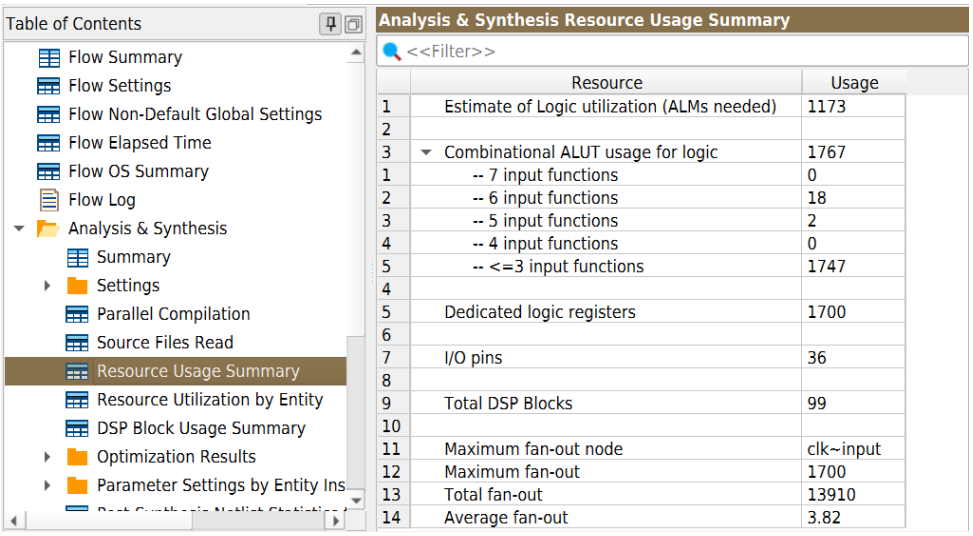
5.2.1 Key Observations

All architectures exhibit an initial transient error during the first ∼100 samples due to pipeline fill, as the Verilog output builds up while MATLAB already has full history. After this phase, the error stabilizes and is dominated by Q(2.14) fixed-point quantization. The Direct and Genvar architectures show similar steady-state error (≈ 2–3×10^{-3}) since both involve 100 multiply-accumulate operations, leading to comparable accumulation of rounding and clipping errors. In contrast, the optimized symmetric architecture exhibits lower error because it performs only $N/2$ operations, reducing the number of quantization steps. Overall, the error magnitude is directly related to the number of arithmetic operations and intermediate truncations.

—

6 Resource Utilization and Timing

6.1 Resource Utilization Summary



	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	1173
2		
3	Combinational ALUT usage for logic	1767
1	-- 7 input functions	0
2	-- 6 input functions	18
3	-- 5 input functions	2
4	-- 4 input functions	0
5	-- <=3 input functions	1747
4		
5	Dedicated logic registers	1700
6		
7	I/O pins	36
8		
9	Total DSP Blocks	99
10		
11	Maximum fan-out node	clk~input
12	Maximum fan-out	1700
13	Total fan-out	13910
14	Average fan-out	3.82

Figure 7: Quartus resource utilization for Direct Form FIR (Non-Pipelined)

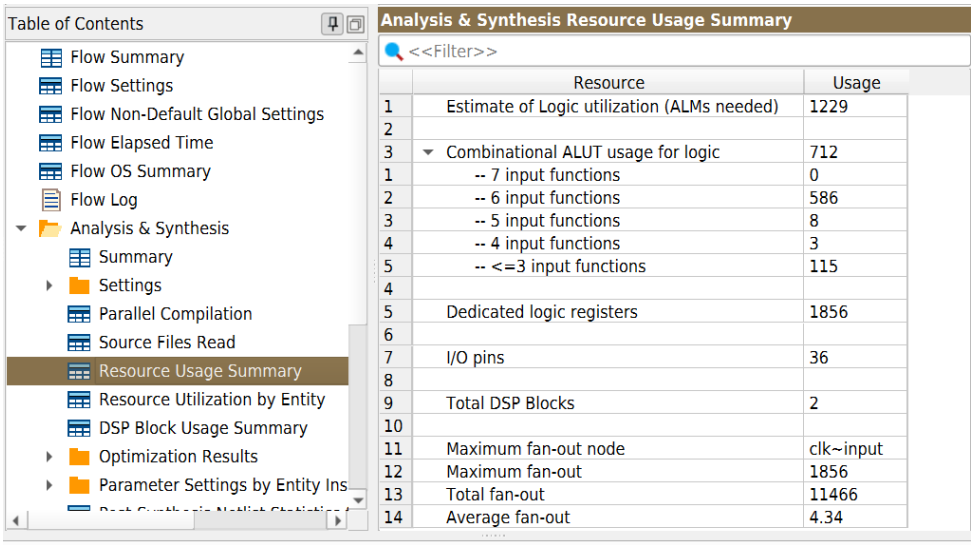


Figure 8: Quartus resource utilization for Optimized Symmetric FIR (Non-Pipelined)

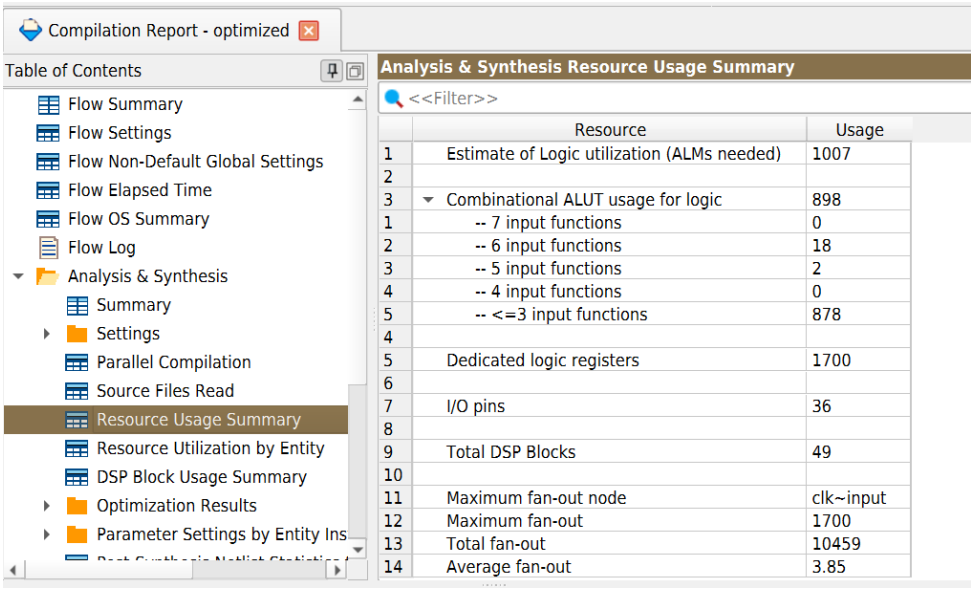


Figure 9: Quartus resource utilization for Genvar Parallel FIR (Non-Pipelined)

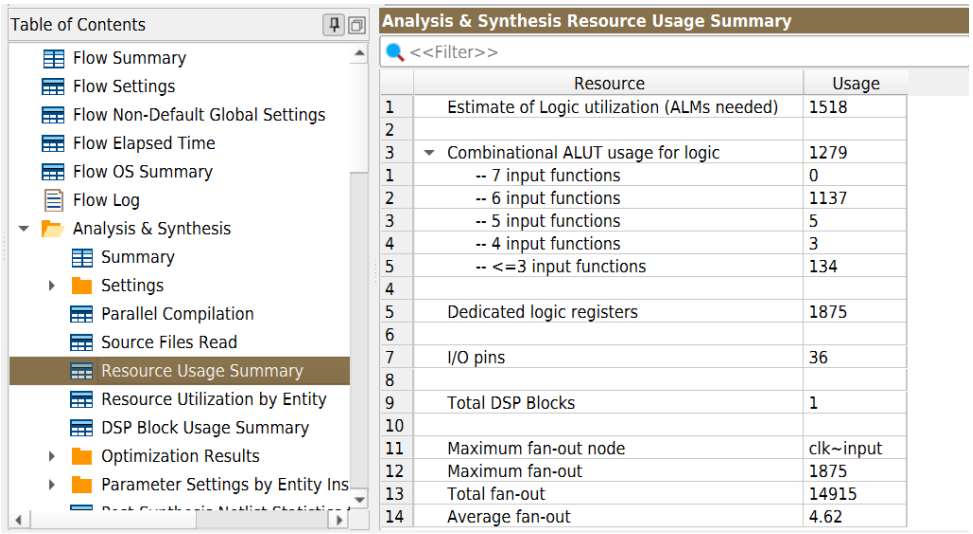


Figure 10: Quartus resource utilization for Direct Form FIR (Pipelined)

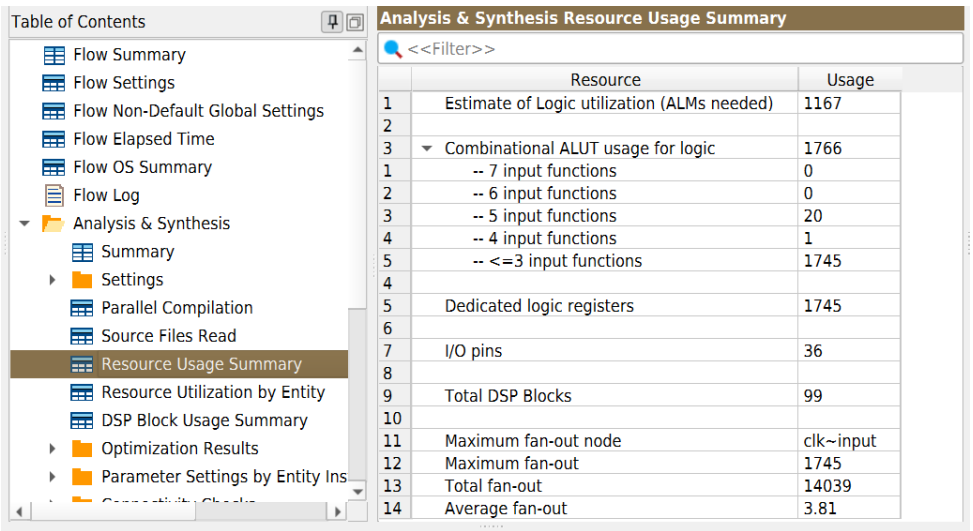


Figure 11: Quartus resource utilization for Optimized Symmetric FIR (Pipelined)

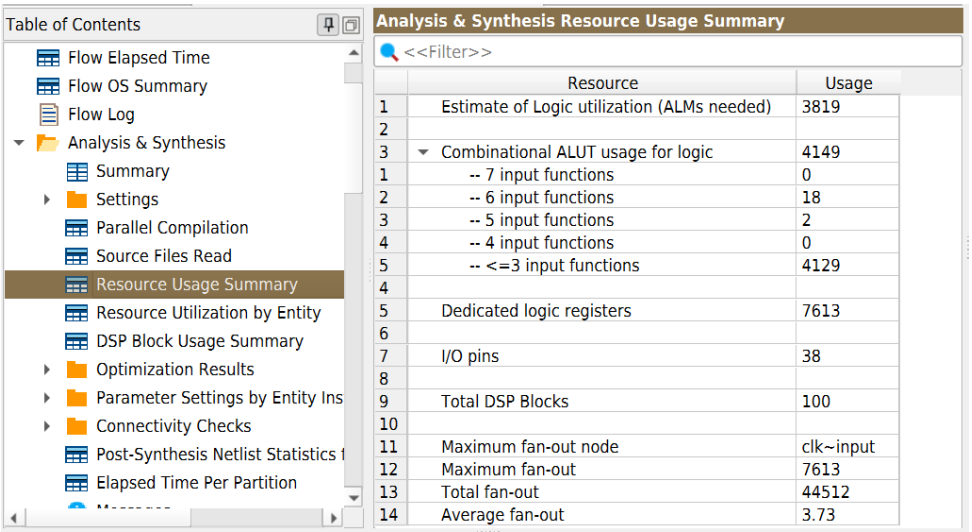


Figure 12: Quartus resource utilization for Genvar Parallel FIR (Pipelined)

6.2 Timing Summary

Architecture	Fmax	Slack	Throughput
Direct Form (Non-Pipelined)	28.36 MHz	-15.975ns	0.284 MHz
Optimized Symmetric (Non-Pipelined)	29.03 MHz	-15.407ns	0.581 MHz
Genvar Parallel (Non-Pipelined)	49.19 MHz	6.008ns	49.19 MHz
Direct Form (Pipelined)	59.52 MHz	3.409ns	0.595 MHz
Optimized Symmetric (Pipelined)	70.64 MHz	6.008ns	1.413 MHz
Genvar Parallel (Pipelined)	66.27 MHz	4.911ns	66.27 MHz

Table 1: Timing comparison of all six FIR implementations.

7 FIR IP Summary

7.1 FIR IP Resource Utilization

Flow Status	In progress - Mon Apr 6 14:03:54 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	FIR_IP
Top-level Entity Name	firip
Family	Cyclone V
Device	5CGXFC7C7F23C8
Timing Models	Final
Logic utilization (in ALMs)	N/A
Total registers	2713
Total pins	63
Total virtual pins	0
Total block memory bits	0
Total DSP Blocks	25
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0
Total DLLs	0

Figure 13: Quartus synthesis report showing resource utilization for FIR IP implementation

7.2 FIR IP Timing Summary

Implementation	Fmax	Slack
FIR IP (Quartus)	64.83MHz	4.576ns

Table 2: Timing summary for Quartus FIR IP implementation.